

TEMA 1:

ARQUITECTURA Y TECNOLOGÍAS DE PROGRAMACIÓN SOBRE CLIENTES WEB

1.1 CARACTERIZACIÓN Y DIFERENCIACIÓN DE LOS MODELOS DE EJECUCIÓN DE CÓDIGO EN EL SERVIDOR Y EN EL CLIENTE WEB

Cualquier aplicación funciona mediante el modelo cliente/servidor, repartiendo el trabajo entre estos a través de internet.

A. Contexto histórico y estandarización de la web.

- **Origen de la Web:** nació en 1989 en el CERN a manos de Tim Berners-Lee.
- **Consortio W3C (World Wide Web Consortium):** Organismo encargado de crear las reglas y estándares oficiales para las páginas web.
- **Nube (Cloud computing):** En la actualidad la mayoría de empresas externaliza o contrata espacio y potencia de cálculo bajo demanda en plataformas de internet como AWS.
- **Especialización del trabajo técnico:**
 - **Diseñadores (UX/UI):** Paleta de colores, tipografía, distribución de la interfaz.
 - **Programadores cliente (Front-end):** Implementan los componentes interactivos, menús y pantallas con las que interactúa el usuario.
 - **Programadores servidor (Back-end):** Desarrollan los servicios para persistir datos, procesar transacciones seguras y proteger accesos.
 - **Administradores de bases de datos (DBA):** Gestionan, optimizan y garantizan la integridad de los datos almacenados.

B. El entorno servidor (Back-end)

El back-end constituye la infraestructura no visible de una aplicación web para el usuario final. Se ejecuta en servidores dedicados o instancias de computación en la nube, el usuario final nunca tiene acceso a esta parte. Sus tareas principales son:

- Autenticar la identidad de los usuarios.
- Realizar cobros y pagos de manera segura.
- Conectarse a bases de datos para almacenar y consultar registros.
 - *BBDD relacionales (SQL): almacenan información en tablas, filas y columnas. Ej: MySQL, MariaDB.
 - *BBDD documentales (NoSQL): Gestionan la información mediante bloques y documentos. Ej: MongoDB.
- Tecnologías y lenguajes habituales en servidor: PHP, Java, Python, Node.js (JavaScript del lado del servidor) o C#(.NET).

C. El entorno Cliente (Front-end)

Representa la capa de presentación que se dibuja directamente en la pantalla del dispositivo del cliente. Se ejecuta localmente dentro del navegador web del usuario. El cliente descarga los archivos a través de la red y el navegador los interpreta utilizando el propio dispositivo local. El código que corre en el cliente es completamente público, pulsando F12 en el teclado se abren las DevTools (herramientas de desarrollador) pudiendo ver y modificar el HTML y JavaScript en tiempo de ejecución. Las claves de cifrado, contraseñas y operaciones contables o de seguridad, jamás deben ubicarse exclusivamente en el cliente. Los tres pilares del front-end son:

- **HTML:** Define la estructura y elementos del documento.
- **CSS:** Diseño y presentación visual.
- **JavaScript:** Permite responder a eventos de ratón/teclado, alterar el dinámicamente el documento y validar información localmente.

D. Tabla comparativa Cliente vs Servidor

Dimensión	Entorno cliente (Front-end)	Entorno servidor (Back-end)
Entorno de ejecución	Navegador del usuario (Chrome, Firefox, Safari, Edge).	Servidor remoto físico o instancia en la nube (AWS).
Visibilidad del código	Público: Accesible e inspeccionable con F12 / DevTools.	Privado: Reside de forma estricta en el servidor.
Tecnologías clave	HTML5, CSS3, JavaScript (React, Vue, Angular).	PHP, Python, Java, Node.js, C# (.NET).
Acceso a datos	Indirecto: Peticiones HTTP a través de internet.	Directo: Conexión nativa a motores SQL o NoSQL.
Consumo de recursos	CPU, memoria RAM y batería del dispositivo local.	Potencia de cómputo y memoria del servidor.
Latencia de respuesta	Inmediata para acciones visuales locales.	Sujeta a la latencia de red y carga del servidor.
Nivel de seguridad	Bajo: Modificable por el cliente mediante scripts.	Alto: Núcleo fiable para cobros, permisos y roles.

E. Evolución de la navegación: De la web tradicional a la web moderna

1. El modelo clásico tradicional:

- Cada clic en un enlace disparaba una petición síncrona al servidor.
- El servidor procesaba la solicitud y ensamblaba un nuevo documento HTML.
- La web sufría una pantalla en blanco y parpadeo al recargarse de nuevo.

2. El modelo moderno con JavaScript (Single Page Applications - SPA):

- La aplicación descarga la plantilla y recursos base una única vez al iniciar.
- Cuando el usuario interactúa con la página, no se recarga completamente.
- JS solicita de forma asíncrona únicamente los datos necesarios empaquetados en **JSON** y actualiza selectivamente las partes del árbol visual que cambian.

F. Criterios de asignación de tareas: Cliente vs. Servidor

Usabilidad e inmediatez va en el cliente; integridad y seguridad va en el servidor.

● Operaciones propias del cliente (Front-end):

- Desplegar y ocultar menús.
- Modificar estilos y colores en respuesta a eventos del ratón.
- Ordenar y filtrar colecciones de datos ya cargados en memoria.

● Operaciones exclusivas del Servidor (Back-end):

- Realizar cobros con pasarelas de pago bancarias.
- Ejecutar operaciones complejas sobre millones de registros en BBDDs.
- Comprobar privilegios de usuario antes de conceder accesos administrativos.

G. Vocabulario técnico

- **Cliente:** Dispositivo y software navegador utilizado por el usuario para interactuar con la aplicación web.
- **Servidor:** Máquina conectada permanentemente a la red que aloja los servicios, gestiona las reglas de negocio y responde a las solicitudes de los clientes.
- **Renderizar:** Proceso computacional mediante el cual el motor del navegador interpreta el código HTML, CSS y JS para pintar los elementos visuales en pantalla.
- **Base de datos:** Sistema informático orientado al almacenamiento estructurado, persistente y consulta ágil de grandes volúmenes de datos.
- **DevTools:** Conjunto de utilidades de diagnóstico integradas en los navegadores (F12) que permiten auditar el DOM, monitorizar peticiones de red y depurar código JS.

1.2. CAPACIDADES Y MECANISMOS DE EJECUCIÓN DE CÓDIGO DE NAV. WEB

A. ¿Qué es un navegador web y cómo se organiza por dentro?

Un navegador web es una aplicación cliente instalada en el dispositivo del usuario cuyo trabajo consiste en pedir páginas a servidores web a través de la red, interpretar el código que recibe y mostrarlo en la pantalla. El interior de un navegador se divide en 7 partes:

1. **Interfaz de usuario:** La ventana con la que interactuamos.
2. **Motor del navegador (*Browser engine*):** Hace de puente entre la interfaz externa y los motores internos, gestionando órdenes como abrir una pestaña nueva, etc.
3. **Motor de renderizado (*Rendering engine*):** Lee el HTML y CSS calculando el tamaño y posición exacta de cada elemento y los dibuja en pantalla.
4. **Motor de JavaScript (*JS engine*):** Intérprete que lee y traduce el código JS a instrucciones máquina y las ejecuta en el procesador empleando compilación **JIT (Just In Time)**, un mecanismo que detecta qué funciones se ejecutan muchas veces y las traduce directamente a código máquina nativo del procesador, ganando velocidad.
5. **Capa de red (*Networking*):** Se encarga de enviar y recibir datos a través de Internet.
6. **Backend de interfaz (*UI backend*):** Conecta el navegador con el SO para dibujar cuadros de texto, ventanas o barras de desplazamiento con el aspecto del sistema.
7. **Almacenamiento de datos (*Data storage*):** Espacio en el disco duro del usuario donde el navegador guarda cookies, archivos en caché y BBDDs locales para que las páginas recuerden información.

B. Motores de navegadores en la actualidad

Navegador	Motor de renderizado	Motor de JavaScript	Empresa
Chrome	Blink	V8	Google
Edge	Blink	V8	Microsoft
Brave/Opera	Blink	V8	Brave SW, Opera
Firefox	Gecko	SpiderMonkey	Mozilla
Safari	WebKit	JavaScriptCore	Apple

C. Proceso de renderizado

Cuando el navegador recibe el archivo HTML, realiza 4 pasos consecutivos para mostrarlo:

1. **Creación del DOM y CSSOM:** El navegador lee el HTML y CSS y crea dos árboles en memoria: DOM para la estructura y CSSOM para los estilos.
2. **Unión en el árbol de renderizado:** Combina el árbol de etiquetas con el de estilos.
3. **Disposición (*Layout*):** El navegador calcula el ancho, alto y coordenadas de cada caja en la pantalla según el tamaño de la ventana.
4. **Pintado:** Dibuja colores, bordes, imágenes, etc., píxel a píxel sobre la pantalla.

Si el navegador encuentra una etiqueta `<script>` mientras lee el HTML, detiene la lectura hasta que el archivo JS se descarga y se ejecuta por completo. Si el script es muy pesado la pantalla se quedará en blanco durante unos instantes.

D. Capacidades nativas del navegador (APIs Web)

El navegador moderno proporciona funciones a las que podemos acceder desde JS:

1. **Manipulación de la página (DOM):** Modificar textos, cambiar colores, ocultar cajas o crear nuevos elementos cuando el usuario pulsa un botón.
2. **Peticiones en segundo plano:** Mediante la instrucción `fetch()`, el navegador puede pedir datos a un servidor web y mostrarlos sin tener que recargar la página.
3. **Guardar datos en el equipo del usuario:**
 - **Cookies:** Textos muy pequeños que se envían al servidor en cada petición.
 - **SessionStorage:** Guarda datos mientras la pestaña siga abierta.
 - **LocalStorage:** Guarda datos de forma permanente en el equipo.
 - **IndexedDB:** Pequeña BBDD del navegador para guardar grandes volúmenes de información cuando la aplicación necesita funcionar sin conexión.
4. **Acceso a dispositivos físicos (siempre con permiso del usuario):**
 - Conocer la ubicación geográfica con la API de geolocalización.
 - Utilizar la cámara y el micrófono para videollamadas.
 - Consultar si el equipo tiene batería suficiente o si está conectado a la red.

E. El DOM (Document Object Model) como puente de comunicación

El DOM es la estructura de la página HTML que el navegador crea para poder trabajar con sus elementos mediante JavaScript. Cada vez que JavaScript cambia el DOM, el motor de renderizado recalcula el espacio de los elementos y vuelve a pintar la pantalla.

F. Mecanismos de salida y comunicación del navegador

JS dispone de 4 vías principales integradas en el navegador para comunicar con el usuario:

1. **Consola de depuración (`console.log()`):** Es un canal directo hacia el panel de diagnóstico del navegador. Se accede mediante la *tecla F12* o *clic derecho > Inspeccionar > Consola*. Los desarrolladores la utilizan para depurar (debug). La consola permite además:
 - Categorizar trazas de error con `console.error()`.
 - Emitir avisos amarillos con `console.warn()`.
 - Medir tiempos exactos de cálculo con `console.time()` y `console.timeEnd()`.
2. **Modificación de contenido HTML (`innerHTML`)**

Propiedad que tienen los nodos del DOM para sobrescribir todo el marcado y texto que contienen. Primero se localiza el elemento mediante su identificador único con `document.getElementById('identificador')` y después se reasigna el contenido.

***Aspecto de seguridad esencial:** Si utilizamos `innerHTML` para meter información que ha escrito un usuario desconocido, un atacante podría escribir una etiqueta `<script>` maliciosa y ejecutar código en el navegador de otras personas (ataque conocido como XSS o Cross-Site Scripting). Para evitar esto se recomienda emplear la propiedad `textContent`.
3. **Flujo directo de marcado**

Método clásico que escribe texto o etiquetas HTML directamente en el flujo de la página mientras el navegador está leyendo.

Si se ejecuta `document.write()` después de que la página haya cargado, el navegador borra de forma irreversible todo el documento HTML existente y deja únicamente lo escrito en esa llamada.

4. Diálogos modales de alerta (`window.alert()`)

Es una función que abre una pequeña ventana emergente nativa del SO con un mensaje y un botón de aceptar. Pertenecce al objeto global `window`, por lo que puede escribirse tanto `window.alert()` como simplemente `alert()`. El diálogo es síncrono y bloqueante.

Hasta que el usuario no pulse el botón “Aceptar” la ejecución del hilo principal de JS se congela por completo.

G. Capacidades prácticas de manipulación dinámica

A través del DOM, JS puede modificar en caliente los tres aspectos visuales de una página:

1. Modificar el contenido de la página web

Permite reescribir bloques informativos al interactuar con botones u otros controles.

2. Cambiar atributos de objetos HTML

El destino `href` de un enlace, el ancho `width` o la ruta `src` de una imagen, se convierte en una propiedad accesible desde JS.

3. Cambiar el estilo CSS en tiempo real

JS puede acceder directamente a las reglas de estilo del elemento a través de la propiedad `.style`. *En JS el guión “-” representa la resta, por ello se usa camelCase para separar propiedades compuestas, ejemplo:

`background-color` en CSS pasa a ser `style.backgroundColor`.

H. El objeto global `window` y el árbol jerárquico del BOM

En el navegador existe una jerarquía fundamental:

BOM (Browser Object Model) frente a DOM (Document Object Model).

- El objeto `window` representa la ventana o pestaña completa del navegador y es el **objeto raíz global** del entorno cliente.
- El `document` (DOM) es en realidad una propiedad que cuelga directamente de `window` (`window.document`).
- **Regla de ámbito global:** `window` contiene muchas funciones y objetos disponibles en la página. Por eso podemos escribir `alert()` en lugar de `window.alert()`.

Ejemplo visual:

```
window
├── alert()
├── console
├── document
└── ...
```

I. Profundización en el renderizado: Reflow (Layout) y repaint

Cuando un script altera el DOM o los estilos, el navegador ejecuta una cadena de operaciones costosas en recursos de la CPU y la tarjeta gráfica:

1. **Reflow (o Re-layout):** Ocurre cuando un cambio de JS altera las **dimensiones geométricas o la posición** de un elemento. El navegador recalcula el espacio que ocupa ese nodo y desplaza a todos los elementos en la página.
2. **Repaint:** Ocurre cuando se modifica una propiedad visual que **no altera el espacio físico**. El navegador no recalcula posiciones, solo repinta los píxeles afectados.

*Los cambios que provocan *reflow* continuos dentro de un bucle ralentizan la página. Las modificaciones visuales deben agruparse para evitar parpadeos y caídas en los fps.

1.3 IDENTIFICACIÓN Y CARACTERIZACIÓN DE LOS PRINCIPALES LENGUAJES RELACIONADOS CON LA PROGRAMACIÓN DE CLIENTES WEB

A. La triada fundamental de la programación cliente

Las páginas web se apoyan directamente sobre tres lenguajes, cada uno con una responsabilidad independiente:

1. HTML (HyperText Markup Language):

- **Naturaleza:** Es un lenguaje de marcado basado en etiquetas.
- **Responsabilidad:** Delimita y define la semántica y estructura del documento.
- **Interpretación:** El navegador web lee e interpreta las etiquetas para representarlas de forma visual.

2. CSS (Cascading Style Sheets):

- **Naturaleza:** Lenguaje declarativo de diseño gráfico y maquetación visual.
- **Responsabilidad:** Define el aspecto, estética, márgenes, etc.
- **Separación de facetas:** No interviene en la lógica ni en los datos de la web; se limita a que la presentación sea atractiva e intuitiva para el usuario.

3. JavaScript:

- **Naturaleza:** Lenguaje de programación dinámico, débilmente tipado y orientado a eventos.
- **Responsabilidad:** Inyecta dinamismo, reacciona a las acciones del usuario, valida entradas y altera la estructura del documento en caliente sin recargar la página.
- El estándar actual de la industria es utilizar **TypeScript**, la cual es una capa sobre JS que le añade **tipado estático**.

CAPA 3: COMPORTAMIENTO Y LÓGICA REACTIVA

JavaScript (ES6+) | Frameworks: ReactJS, Angular, Vue.js

- Manipula el árbol DOM en memoria.
- Gestiona el flujo de datos y el estado de la aplicación.

CAPA 2: PRESENTACIÓN Y ESTILO VISUAL

CSS3 | Preprocesadores / Frameworks UI

- Colores, tipografías, maquetación flexible (Flexbox, Grid).
- No altera la estructura del contenido ni la lógica de negocio.

CAPA 1: ESTRUCTURA SEMÁNTICA Y CONTENIDO

HTML5 (Lenguaje de Marcado de Hipertexto)

- Define qué elementos existen (encabezados, párrafos, formularios).
- Base universal estandarizada por el W3C para cualquier navegador.

B. JavaScript Evolución histórica

1. **El origen: 10 días en Netscape (1995)**
JS nació en mayo de 1995 cuando la web estaba dominada por el navegador Netscape.
2. **El nacimiento y la estandarización temprana (1995–1999)**
 - **1995:** Brendan Eich crea el lenguaje en 10 días para Netscape.
 - **1996:** Microsoft lanza Internet Explorer 3.0 con una versión propia llamada JScript.
 - **1997:** nace ECMAScript.
 - **1998 y 1999:** ES3 consolida el lenguaje durante la siguiente década.
3. **El estancamiento y la era AJAX (2000–2008)**
 - **Fracaso de ES4** por su complejidad.
 - **Auge de AJAX (2005)**, Jesse James Garrett acuñó el término. El uso de XMLHttpRequest permite actualizar páginas web sin recargarlas por completo, transformando a JS en una herramienta de aplicaciones web completas (Google Maps, Gmail).
 - **Las librerías de abstracción (2006):** Se crean librerías como *jQuery*, *Prototype* y *MooTools* para unificar las APIs del DOM y mitigar las inconsistencias entre navegadores.
4. **La madurez: ES5 y la salida del navegador (2009)**
En 2008 nació Harmony que dio lugar a:
 - **ECMAScript 5 (diciembre 2009):** Modo estricto, métodos funcionales de arrays, soporte nativo para JSON, Getters y Setters.
 - **El motor V8 y Node.js (2008-2009):** Google lanza Chrome con el motor V8 y **Ryan Dahl crea Node.js sobre V8.**
5. **El punto de inflexión: ECMAScript 2015 (ES6)**
Supuso la mayor refundición de la sintaxis y capacidades del lenguaje desde su creación.
6. **La era moderna: Lanzamientos anuales (ES2016+)**
A partir del ES6, el TC39 adoptó un proceso de publicaciones anuales.

C. El ecosistema de framework y librerías de front-end

Las aplicaciones no se suelen construir utilizando solo JS sino apoyándose en un framework o librería avanzada.

- **Origen:** Nacieron como librerías de funciones para simplificar tareas repetitivas.
- **Evolución:** Hoy constituyen plataformas de desarrollo completas (compilación, lenguajes tipados, TypeScript).
- **Ventajas que aportan los frameworks al desarrollo empresarial:**
 - Coste económico nulo.
 - Fiabilidad, seguridad y rendimiento.
 - Velocidad de entrega.
 - Estandarización de equipos.

D. Caracterización de los principales frameworks del mercado

1. **ReactJS:** Librería creada por Meta cuya programación está orientada a componentes. Utiliza DOM Virtual y usa Sintaxis JSX.
2. **Angular:** Plataforma de desarrollo de Google. Su primera versión fue AngularJS. Se programa en TypeScript. Presenta una curva de aprendizaje pronunciada debido a su rigidez estructural.
3. **Vue.js**
Diseñado por Evan You con la premisa de tomar las mejores características de React y Angular. Emplea también un DOM virtual para optimizar el renderizado. Curva de aprendizaje progresiva.
4. **Otros frameworks y librerías alternativas**
EmberJS, BackboneJS, MeteorJS, Aurelia.js, Polymer y Mithril.js

E. Vocabulario

- **DOM Virtual:** Consiste en que el framework guarde en la memoria RAM una copia del DOM original, reduce las renderizaciones en el navegador incrementando el rendimiento.
- **JSX:** Extensión de JS parecida a un lenguaje de plantillas pero con toda la capacidad de ejecución de JS integrado.
- **TypeScript:** Lenguaje de código abierto desarrollado por Microsoft, actúa como un superconjunto de JS que añade tipado estático para grandes proyectos y se compila a JS ejecutable estándar.
- **Patrón reactivo:** Modelo de programación basado en flujos de datos asíncronos que reacciona de forma automática propagando los cambios en la interfaz cuando el estado de los datos varía.

F. Funciones en JS

Una función en JS es un bloque de código reutilizable diseñado para realizar una tarea específica. Se define una vez y puede invocarse tantas veces como se necesite.

Formas principales de declararlas:

- **Declaración tradicional (*Function declaration*):** Es la forma clásica, disfruta de *hoisting*, es decir, se puede invocar antes de la línea donde está escrita. Ej:

```
function saludar(nombre) {  
  return `Hola, ${nombre}`;  
}  
  
console.log(saludar("Ana")); // "Hola, Ana"
```

- **Expresión de función (*Function Expression*):** Se asigna una función a una variable o constante. No se puede usar antes de definirla. Ej:

```
const duplicar = function(numero) {  
  return numero * 2;  
};  
  
console.log(duplicar(5)); // 10
```

- **Funciones flecha (*Arrow Functions - ES6*):**

Sintaxis moderna y compacta. Si el cuerpo tiene una sola línea, el `return` y las llaves `{}` son implícitos.

```
// Retorno implícito  
const sumar = (a, b) => a + b;  
  
// Si recibe un solo parámetro, se pueden omitir los paréntesis  
const cuadrado = x => x * x;  
  
console.log(sumar(3, 4)); // 7
```

1.4 PARTICULARIDADES DE LA PROGRAMACIÓN DE GUIONES (scripts) Y SUS VENTAJAS Y DESVENTAJAS SOBRE LA PROGRAMACIÓN TRADICIONAL

A. Origen, concepto y naturaleza de los scripts

- **Nacimiento de los scripts:** Surgieron como secuencias de comandos diseñados para automatizar tareas rutinarias y repetitivas en los sistemas operativos.
- **Dependencia del intérprete:** Los scripts siempre son ejecutados por un intérprete de comandos.
- **De pequeñas macros a programas complejos:** En la web actual representan programas con arquitecturas complejas de miles de líneas código.

B. Diferencias fundamentales entre lenguajes de script y lenguajes tradicionales

1. Compilación frente a interpretación

- *Lenguajes tradicionales*: necesitan compilar el código antes de ejecutarlo. La compilación transforma el código fuente en código que el ordenador puede ejecutar, normalmente generando un archivo ejecutable.
- *Lenguajes de script*: utilizan un intérprete o motor que procesa y ejecuta el código durante su ejecución, sin que el programador tenga que generar previamente un ejecutable independiente.

2. Ejecución independiente frente a integración en un entorno (Host)

- *Lenguajes tradicionales*: generalmente permiten crear programas independientes (standalone) que se ejecutan directamente en el sistema operativo. Por ejemplo, C++ puede generar archivos `.exe`.
- *Lenguajes de script*: tradicionalmente necesitan un entorno que permita ejecutarlos, llamado Host. Por ejemplo, JavaScript puede ejecutarse dentro de un navegador web. Sin embargo, también pueden ejecutarse desde una terminal o convertirse en aplicaciones independientes mediante herramientas adicionales.

3. Desarrollo desde cero frente a reutilización de componentes

- *Lenguajes tradicionales*: suelen crear sus propias estructuras y utilizar sus propias librerías para desarrollar los programas.
- *Lenguajes de script*: están diseñados para aprovechar componentes y funcionalidades que ya proporciona el entorno donde se ejecutan. Por ejemplo, JavaScript utiliza los elementos HTML y las funcionalidades del navegador.

4. Momento de detección de errores

- *Lenguajes tradicionales*: durante la compilación se detectan muchos errores de sintaxis y de tipos. Si existen errores que impiden compilar, no se genera el ejecutable.
- *Lenguajes de script*: muchos errores se detectan durante la ejecución del programa (runtime), cuando el intérprete o motor encuentra la instrucción problemática. Por eso es importante realizar pruebas.

5. Clasificación de los lenguajes

- *Lenguajes de programación tradicional*: C, C++, Java, Swift, Pascal.
- *Lenguajes de scripting*: JavaScript, Shell script, Perl, PHP, Python, Ruby.

C. Análisis de ventajas y desventajas de la programación de guiones

Ventajas:

- Sencillez y curva de aprendizaje rápida.
- Agilidad en el ciclo de desarrollo.
- Integración natural.
- Portabilidad mediante el anfitrión.

Desventajas:

- Mayor tasa de errores en tiempos de ejecución.
- Rendimiento bruto inferior.
- Exposición del código fuente.

D. Casos singulares y proyección del ecosistema de Scripting

- La proyección de Python en el mercado al ser el lenguaje de referencia en desarrollo de IA, computación científica y tratamiento masivo de datos.
- El caso de Java vs. JavaScript, lenguajes con filosofías opuestas:
 - **Java**: Lenguaje programación, fuertemente tipado, compilado a bytecode, orientado rígidamente a objetos y ejecutable en una máquina virtual.
 - **JavaScript**: Lenguaje de script, dinámico, débilmente tipado, interpretado directamente en el navegador y orientado a eventos.

E. El objeto `Date` en JavaScript

1. Naturaleza y funcionamiento de las fechas

En JavaScript, las fechas no son datos primitivos como los números o los textos, sino que son objetos creados a partir de `Date`.

La fecha es una instantánea: cuando creas un objeto `Date`, guarda la fecha y hora de ese momento. No se actualiza automáticamente como un reloj.

Época Unix (Epoch Time): JavaScript representa internamente las fechas mediante los milisegundos transcurridos desde el 1 de enero de 1970 a las 00:00 UTC. Por ejemplo:

```
const tiempoActual = Date.now();
console.log(tiempoActual);
```

Devuelve un número que representa los milisegundos transcurridos desde 1970 hasta el momento actual.

- 1 segundo = 1.000 milisegundos.
- 1 día = 86.400.000 milisegundos.

Los números positivos representan fechas posteriores a 1970 y los negativos, anteriores.

2. Formas de crear fechas (`new Date`)

Existen cuatro formas principales de crear un objeto `Date`:

- **Fecha y hora actuales:**

```
const ahora = new Date();
```

Guarda la fecha y hora exactas del momento en que se ejecuta.

- **Mediante texto:**

```
const fecha = new Date("2026-09-28");
```

Permite indicar una fecha mediante una cadena de texto. Se recomienda utilizar el formato ISO (AAAA-MM-DD).

- **Mediante números:**

```
const fecha = new Date(2026, 8, 28);
```

Permite indicar año, mes y día, además de horas, minutos y segundos si se necesitan. Hay dos reglas importantes:

- Los meses empiezan en 0: enero = 0, febrero = 1... diciembre = 11.
- Los días empiezan en 1: del 1 al 31.

JavaScript también ajusta automáticamente los valores que se salen de los límites. Por ejemplo, si indicamos el día 35 de junio, calculará la fecha correspondiente de julio.

- **Mediante milisegundos:**

```
const fecha = new Date(0);
```

Interpreta el número como milisegundos transcurridos desde el 1 de enero de 1970. Por ejemplo, `new Date(86400000)` representa un día después del inicio de la época Unix.

`new Date(2026)` → 2026 milisegundos después de 1970.

`new Date(2026, 0, 1)` → 1 de enero de 2026.

3. Métodos principales de conversión y salida

Método	¿Qué devuelve?
<code>toString()</code>	Fecha y hora completas, con zona horaria.
<code>toDateString()</code>	Solamente la fecha.
<code>toTimeString()</code>	Solamente la hora y zona horaria.
<code>toISOString()</code>	Fecha y hora en formato ISO y UTC.
<code>toUTCString()</code>	Fecha y hora en formato UTC.
<code>toLocaleDateString()</code>	Fecha adaptada al idioma y región del usuario.

Ejemplo:

```
const fecha = new Date();

console.log(fecha.toDateString());
console.log(fecha.toLocaleDateString("es-ES"));
```

La primera muestra la fecha en un formato de texto estándar y la segunda utiliza el formato español. La idea es sencilla: todos estos métodos muestran la misma fecha, pero con formatos diferentes.

4. Métodos para consultar y modificar fechas (Getters y Setters)

Estos métodos permiten acceder a una parte concreta de la fecha o cambiarla.

Getters: Obtienen información de una fecha sin modificarla.

Método	¿Qué obtiene?
<code>getFullYear()</code>	Año.
<code>getMonth()</code>	Mes (0-11).
<code>getDate()</code>	Día del mes (1-31).
<code>getDay()</code>	Día de la semana (0-6).
<code>getHours()</code>	Hora.
<code>getMinutes()</code>	Minutos.
<code>getSeconds()</code>	Segundos.
<code>getTime()</code>	Milisegundos desde 1970.

Ejemplo:

```
const fecha = new Date(2026, 8, 28);

console.log(fecha.getFullYear()); // 2026
console.log(fecha.getMonth());    // 8
console.log(fecha.getDate());      // 28
```

Setters: Permiten cambiar una parte de la fecha.

Método	¿Qué modifica?
--------	----------------

<code>setFullYear()</code>	Año.
----------------------------	------

<code>setMonth()</code>	Mes.
-------------------------	------

<code>setDate()</code>	Día del mes.
------------------------	--------------

Ejemplo:

```
const fecha = new Date(2026, 8, 28);

fecha.setFullYear(2027);
fecha.setMonth(0);
fecha.setDate(15);
```

Después de ejecutar las tres instrucciones, la fecha será el 15 de enero de 2027.

1.5 VERIFICACIÓN DE LOS MECANISMOS DE INTEGRACIÓN DE LOS LENGUAJES DE MARCAS CON LOS LENGUAJES DE PROGRAMACIÓN DE CLIENTES WEB

A. La etiqueta `<script>` y su evolución técnica

La etiqueta `<script>` es el contenedor oficial para insertar o enlazar código ejecutable dentro de un documento HTML:

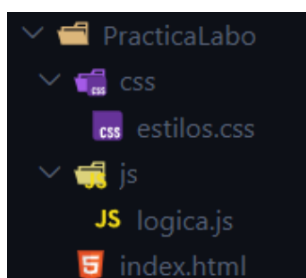
- **Sintaxis actual (HTML5):** Basta con abrir y cerrar la etiqueta `<script>...</script>`.
- **Compatibilidad histórica:** En versiones anteriores era obligatorio especificar el tipo MIME mediante el atributo `type`: `<script type="text/javascript">` aunque los navegadores actuales lo sigan reconociendo, ya no es necesario usarlo.
- **La sintaxis estricta de cierre:** Una etiqueta `<script>` debe incluir obligatoriamente su etiqueta de cierre `</script>`.

B. Código JavaScript en ficheros externos separados

Consiste en mantener la estructura HTML en un archivo con extensión `.html` y extraer toda la lógica de programación a archivos de texto independientes con extensión `.js`.

Ventajas de utilizar ficheros externos:

- **Velocidad de carga y aprovechamiento de la memoria caché:** El navegador descarga el archivo `.js` una vez y lo almacena en su caché pudiendo usarlo en otras páginas de la misma web que usen ese script.
- **Independencia de facetas (modularidad):** Al separarse HTML de JavaScript, permite que diseñadores y programadores trabajen simultáneamente sin pisarse el código.
- **Mantenimiento y reutilización del código:** Se modifica un solo archivo `.js` y los cambios se reflejan en todas las páginas web que lo referencian.
- **Buenas prácticas de ordenación de carpetas:** Empresas y desarrolladores organizan los recursos situando los scripts dentro de un directorio dedicado. Ejemplo visual:



C. Código JavaScript Embebido dentro del HTML

Permite insertar bloques de código JavaScript directamente entre las líneas de marcado del propio archivo HTML. Características y desventajas de este enfoque:

- **Mismo resultado visual:** Tanto el enfoque embebido como el externo provocan exactamente el mismo resultado.
- **Dificultad de mantenimiento:** Bloques `<script>` diseminados a lo largo del `<head>` y `<body>`
- **Criterio de uso excepcional:** Cuando las líneas de código son mínimas.

D. Reglas de ubicación ¿dentro de `<head>` o dentro de `<body>`?

El código JavaScript puede situarse tanto en el `<head>` como en el `<body>` pero la posición influye de manera determinante en cómo se procesa la página web:

- **Ubicación en el `<head>`:**
 - El navegador lee el documento de arriba a abajo. Si encuentra una etiqueta `<script>` en el `<head>`, detiene el análisis del HTML hasta que el script se descarga y se ejecuta por completo.
 - *Problema común:* Si ese script intenta acceder a un elemento del `<body>` (por ejemplo, mediante `document.getElementById('prueba')`), fallará con un error porque ese elemento aún no ha sido leído ni construido en el DOM.
- **Ubicación al final del `<body>` (Antes de `</body>`):**
 - Recomendada ya que garantiza que todo el marcado HTML, textos e imágenes se analicen e inserten en el DOM antes de que se ejecute la lógica de interacción.
- **Profundización moderna: Atributos `defer` y `async` (HTML5):**
 - Para scripts externos colocados en el `<head>`, los estándares modernos permiten evitar el bloqueo del navegador mediante dos atributos:
 - **`defer`:** Descarga el archivo JavaScript en segundo plano mientras el navegador sigue construyendo el HTML, pero retrasa su ejecución exacta hasta que el documento HTML se ha parseado por completo.
Ejemplo: `<script defer src="script.js"></script>`
 - **`async`:** Descarga el archivo en segundo plano y lo ejecuta de inmediato en cuanto termina la descarga, sin importar si el HTML ha terminado de leerse.
Ejemplo: `<script async src="script.js"></script>`

1.6 RECONOCIMIENTO Y EVALUACIÓN DE LAS HERRAMIENTAS DE PROGRAMACIÓN Y PRUEBA SOBRE CLIENTES WEB

A. Herramientas locales: De editores básicos e IDEs avanzados

Ranking de editores top para JavaScript/TypeScript:

1. **Visual Studio Code (75,9% de uso global):** Más del 80% de los desarrolladores front-end lo usan como su herramienta principal.
2. **Notepad++ (27,4% de uso global):** Usado en Windows para la edición rápida de scripts sueltos de JavaScript, manipulación veloz de archivos .json o tareas ligeras de automatización sin consumir recursos.
3. **Vim/Neovim (38,3% de uso combinado):** Mismo autocompletado y tipado inteligente de TypeScript que ofrece VS Code, pero corriendo directo en la terminal a máxima velocidad.
4. **Cursor (17,9% de uso global y subiendo):** Permite usar la IA nativa para generar componentes interactivos de interfaces web o refactorizar archivos TypeScript complejos con instrucciones simples en lenguaje natural.
5. **Los IDEs de JetBrains (15,1% combinados en web):** Cotizado en entornos profesionales y corporativos debido a que su motor de refactorización de código y detección de rutas rotas en JavaScript/TypeScript es el más inteligente y seguro del mercado.

Características técnicas fundamentales para la elección de un entorno profesional:

- **Código abierto y gratuidad.**
- **Arquitectura modular.**
- **Gestor de paquetes integrado.**
- **Autocompletado predictivo.**
- **Sistema de paneles múltiples.**
- **Soporte y canales comunitarios.**

B. Integración con Sistemas de Control de Versiones (Git y GitHub)

El desarrollo en equipo exige registrar el historial de cambios, gestionar ramas de características y coordinar modificaciones concurrentes:

- **Git:** Sistema que rastrea cada modificación realizada en los archivos del proyecto a lo largo del tiempo.
- **GitHub:** Plataforma en la nube para alojar repositorios Git, facilitando la revisión de código por pares (pull request), el seguimiento de incidencias (issues) y la integración continua.
- **Integración en IDEs:** Editores como VSC integran paneles nativos de Git que permiten confirmar cambios (commit), alternar entre ramas y resolver conflictos sin salir del entorno de edición.

C. Entornos de programación y prueba online

- **Coding Ground (Tutorialspoint):** Proporciona un editor con resaltado de sintaxis, visualización previa y consola de ejecución simultánea. Permite la gestión de múltiples ficheros dentro de un mismo proyecto, etc.
- **Otras plataformas en la nube:** CodeSandbox, StackBlitz o JSFiddle permiten evaluar librerías y componentes sin requerir instalación previa, arrancando proyectos de React, Angular o Vue directamente desde el navegador en un par de segundos.

D. Herramientas de prueba y depuración del navegador (DevTools)

- **Panel Consola (Console):** Permite ejecutar JavaScript en tiempo real y visualizar los mensajes de `console.log()` y los errores.
- **Panel Fuentes (Sources / Debuggers):** Permite examinar el código JavaScript, establecer puntos de interrupción (breakpoints) y analizar paso a paso las variables y la ejecución del programa.
- **Panel Red (Network):** Permite supervisar las peticiones HTTP de la página, comprobar los códigos de respuesta (200, 404, 500), el tiempo de carga y el tamaño de los archivos.

E. Criterios de evaluación y selección de herramientas

Parámetro de Decisión	Editor Ligero / Online	IDE Completo / Avanzado
Escenario de uso idóneo	Pruebas de concepto rápidas, corrección puntual de errores, equipos con hardware limitado	Proyectos profesionales medianos y grandes, aplicaciones basadas en frameworks (React, Angular).
Consumo de recursos	Mínimo; funciona en cualquier navegador web.	Medio-alto; requiere memoria RAM y almacenamiento local para indexar el proyecto.
Control de versiones	Limitado a exportar o descargar archivos sueltos.	Integración profunda con Git, ramas, control de diferencias visuales y GitHub.
Personalización	Escasa o nula; depende de la plataforma web.	Elevada; personalizable mediante gestores de paquetes y extensiones de la comunidad.

2.1 SINTAXIS DEL LENGUAJE JAVASCRIPT

Evolución y selección de tecnologías en cliente

1. **La era oscura:** A finales de los 90 y principios de los 2000, los navegadores eran muy limitados dependiendo de *plugins* externos para crear páginas interactivas o juegos. Ej: *Java Applets, Adobe Flash, VBScript*.
2. **La consolidación: Estándar universal**
Los navegadores modernos entienden un único lenguaje nativo: JavaScript.
3. **La modernidad: Superoferta y dialectos**
La industria creó herramientas para mejorar la experiencia de desarrollo con la condición de que todo se tradujera a JavaScript puro para que el navegador lo entienda.
 - **TypeScript:** Añade tipado estático a JS obligando a definir si una variable es texto, número o fecha evitando así errores en el código.
 - **JSX:** Sintaxis que permite mezclar código HTML directamente dentro de JS. Es la pieza clave de librerías como React para crear componentes visuales reutilizables.

A. Razones técnicas para la selección de JS en Front-end

1. Estandarización e interoperabilidad sin extensiones (Zero plugins).
2. Reducción de la carga de cálculo y tráfico en el servidor.
3. Modelo asíncrono y no bloqueante (Orientado a eventos).
4. Ecosistema universal (Full-stack).

B. Reglas sintácticas universales del lenguaje

1. **El punto y coma (;)**
 - Se utiliza para separar sentencias.
 - Después de usarlo es de buena práctica y aconsejable hacer un salto de línea para evitar ambigüedades y malas interpretaciones en el motor.
2. **Sensibilidad a mayúsculas y minúsculas (Case-sensitive)**
JS distingue entre mayúsculas y minúsculas, por ejemplo, si escribimos `Alert()` en vez de `alert()` el script producirá un error sintáctico o de referencia en tiempo de ejecución.
3. **Reglas obligatorias de los nombres de identificadores (variables, funciones, objetos)**
 - Comenzar por una letra (a-z, A-Z), dólar (\$) o guión bajo (_).
 - Nunca comenzar por un número.
 - No pueden utilizarse palabras reservadas del lenguaje (`break`, `case`, `catch`)
4. **Conjunto de caracteres y codificación**
JS utiliza **Unicode** permitiendo emplear cualquier carácter o símbolo en cadenas y textos.

El conflicto del punto y coma: ASI (*Automatic Semicolon Insertion*)

JS inserta automáticamente (;) cuando detecta que una instrucción ha terminado.

Problema: puede interpretar incorrectamente algunos saltos de línea, provocando errores. Por ejemplo, si ponemos un salto de línea después de `return`, puede insertar un punto y coma y hacer que la función devuelva `undefined`.

Convenciones de nomenclatura profesional (Clean code)

- **camelCase:** Estándar para variables, propiedades y funciones. Ej: `calcularImporte`
- **PascalCase (o UpperCamelCase):** Reservado para nombres de clases, componentes (React) y funciones constructoras. Ej: `UsuarioRegistrado`
- **UPPER_SNAKE_CASE:** Constantes globales inmutables cuyos valores se conocen en tiempo de diseño (`MAX_REINTENTOS`, `PI`, `URL_API`)
- **Prefijo \$:** Común en librerías para referenciar elementos directos del DOM (`const $botonEnviar = ...`)
- **Prefijo _:** Para denotar propiedades o métodos con intención privada dentro de objetos o clases.

C. Especificación del lenguaje: De JavaScript a ECMAScript (ES6 / ES2015)

El salto de ES6 marcó la transición hacia el desarrollo profesional moderno mediante la introducción de características como:

- Declaración de variables con ámbito de bloque (**let** y **const**), corrigiendo las fugas de ámbito **var**.
- Sintaxis formal de clases (class), facilitando la orientación a objetos.
- Funciones flecha (**=>**) para una sintaxis más limpia y mejor manejo del contexto **this**.
- Métodos declarativos avanzados para matrices (**map**, **filter**, **reduce**).

D. Evaluación de alternativas: JavaScript frente a TypeScript y JSX

Criterio de selección	JS nativo (Vanilla JS)	TypeScript	JSX (JavaScript XML)
Definición	Lenguaje estándar nativo interpretado por navegadores.	Superconjunto tipado de código abierto creado por Microsoft.	Extensión de sintaxis para React parecida a plantillas con JS integrado.
Tipado de datos	Débil y dinámico (las variables cambian de tipo en ejecución).	Fuerte y estático (se declaran tipos para variables, parámetros y retornos).	Hereda el tipado de JavaScript subyacente.
Fase de compilación	Ninguna; se descarga y ejecuta directamente en el navegador.	Obligatoria; se compila a JavaScript estándar para poder ejecutarse.	Obligatoria; herramientas como Babel la transforman en llamadas JS convencionales.
Detección de errores	En tiempo de ejecución (runtime).	En tiempo de desarrollo y compilación previa.	Al compilar el árbol de componentes.
Proyecto recomendado	Scripts de tamaño pequeño a medio, manipulación directa del DOM.	Aplicaciones corporativas de gran escala (estándar en Angular).	Aplicaciones orientadas a componentes en ReactJS.

E. Glosario técnico asociado a la selección del lenguaje

- **Case-sensitive:** Indica que el lenguaje es sensible a mayúsculas y minúsculas.
- **Consola de JavaScript:** Panel de diagnóstico técnico accesible comúnmente con la tecla F12 que permite comunicar el script con el exterior mediante `console.log()` para verificar variables y depurar fallos.
- **Transpilador:** Compilador específico que traduce código fuente escrito en un lenguaje o dialecto moderno (como TypeScript o ES2020) a código fuente JavaScript estándar compatible con navegadores más antiguos (ejemplo: Babel o el compilador de TypeScript `tsc`).